

▶システム開発

📖システム開発の流れ

📖システム開発手法

📖業務モデリングとユーザーインターフェース

📖モジュール分割

📖テスト手法

- 用語集

用語	フルネーム	意味
		1. 企画プロセス
共通フレーム	-	2. 要件定義プロセス 3. 開発プロセス 4. 運用プロセス 5. 保守プロセス
RFI	Request For Information	情報提供依頼書。一般的な情報の提供を求める（=一次審査）
RFP	Request For Proposal	提案依頼書。個別の具体的な提案を求める（=二次審査）
V字モデル	-	設計工程とテスト工程が紐づいている 1. 最初に要件を定義する 2. システムを作る設計をする 3. コーディングを行う 4. 細かい単位でテスト 5. システム全体のテスト 6. 納品のチェック
レビュー	-	工程ごとに成果物の検証を行う
ウォークスルー	Walk Through	担当者が関係者に対して成果物の内容を説明する
インスペクション	Inspection	モデレーターが司会進行役となり、参加者の役割を明確にして進める
ラウンドロビン	Round Robin	参加者全員がテーマごとに持ち回りで責任者となり進める

用語	フルネーム	意味
EA	-	エンタープライズアーキテクチャ 一つ一つのシステムを最適化するのではなく、企業全体で見た時の最適化を目指す 1. ビジネスアーキテクチャ 2. データアーキテクチャ 3. アプリケーションアーキテクチャ 4. テクノロジーアーキテクチャ
ウォーターフォール	-	上流から下流へ順番に工程を進める
アジャイル	-	(機敏な、素早い) 短い開発サイクルを回す
XP	eXtreme Programing	エクストリームプログラミング 顧客の要望を取り入れながら柔軟に開発を進める 開発時の習慣的な手法5つの価値と19のプラクティスが定められている
ペアプログラミング	-	コードを書く人（ドライバー）とコードをチェックする人（ナビゲーター）の役割に分担して、2人1組でプログラミングを行う
リファクタリング	-	外部からの挙動は変えずに、プログラムの内部構造を変更する
テスト駆動開発	-	プログラム実装前に先にテストコードを書いて、そのテストを通るようにコードを書く
スクラム	-	開発チームが一体となり短期間で開発を進める スプリントと呼ばれる任意から4週間程度の短い開発サイクルを回す
プロトタイプングモデル	-	試作品を作成し、利用者がチェックを行いながら進める
リバースエンジニアリング	-	開発工程を遡り、設計書を取り出す
E-R図	Entity-Relationship Diagram	データベース設計で使用され、実体（対象業務を構成するモノ）と実体の関連をモデル化したもの
DFD	Data Flow Diagram	システムにおけるデータの流れをモデル化したもの
UML	-	オブジェクト指向のシステム開発で使用され、分析から設計、実装まで一貫した表記方法でモデル化したもの
ユーザーインターフェース	UI	ユーザーとコンピューターの接点となるもの。ユーザーとコンピューター間で情報をやり取りする機械や入力装置
画面設計	-	画面の項目やレイアウトなど、データを入力する際の操作方法を設計

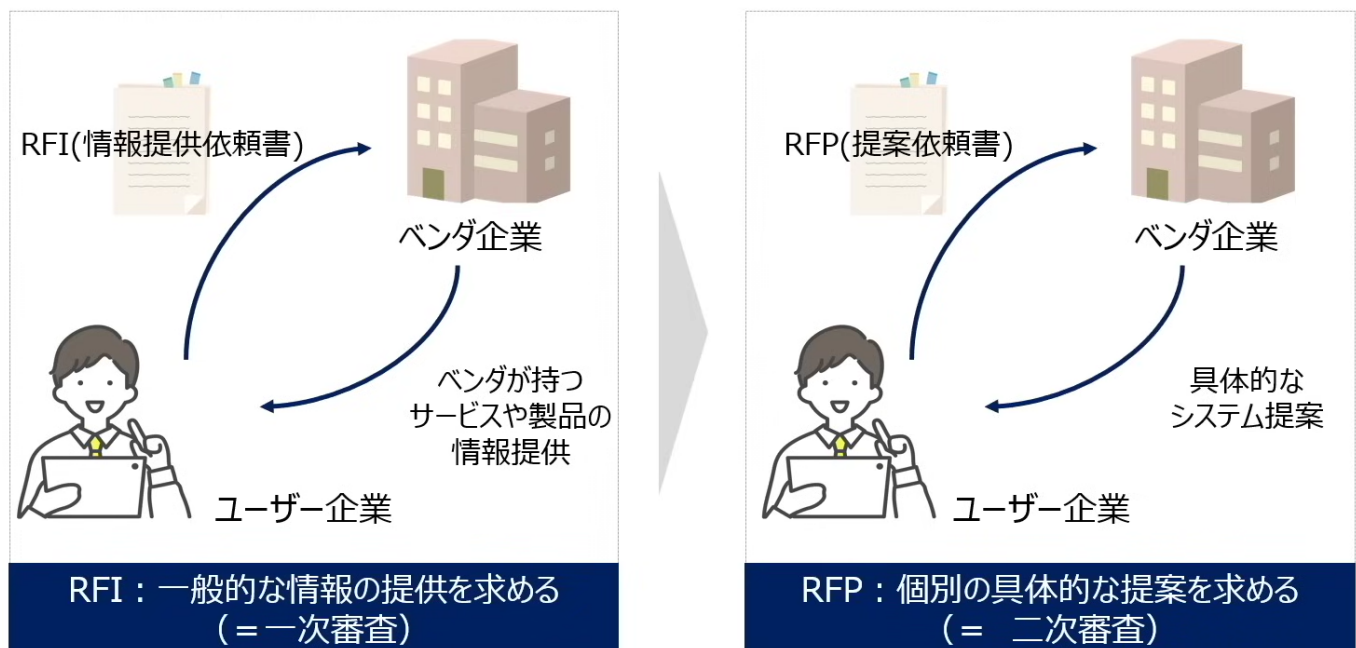
用語	フルネーム	意味
帳票設計	-	帳簿や伝票などのレイアウト、プリンターに出力する内容などを設計
GUI	Graphical User Interface	グラフィカルユーザーインターフェース 画面上のアイコンやボタンなどをマウス操作する視覚的インターフェース
CUI	Character User Interface	キャラクターユーザーインターフェース キーボードによる文字入力を用いたインターフェース 画像やアイコンを使わず文字だけで操作し、テキストデータのみで表現される
モジュール分割	-	内部設計の段階で行う システム全体をモジュール単位で分割を行い、モジュール単位で開発・テストを行っていく
モジュールの独立性	-	独立性が高いとシステムとメンテナンスが行いやすい モジュール結合度が弱い方が、モジュールの独立性が高くなり、システムメンテナンスしやすい モジュール強度が強い方が、モジュールの独立性が高くなり、システムメンテナンスがしやすい
モジュール結合度	-	1. データ結合：処理に必要なデータだけを引数として渡す 2. スタンプ結合：データ構造（データのまとまり）を引数として渡す 3. 制御結合：モジュールの実行順序を制御するデータ（制御パラメータ）を引数として渡す 4. 外部結合：外部宣言されたデータを参照する 5. 共通結合：外部宣言されたデータ構造（データのかたまり）を参照する 6. 内容結合：他のモジュール内のデータを参照する
モジュール強度	-	1つのモジュールが1つの機能に洗練（強化）されているか 1. 機能的強度：一つの機能のみを持つモジュール 2. 情報強度：同じデータを使う複数の機能をまとめたモジュール 3. 連絡的強度：データの受け渡し・参照をしながら、順番に実行される複数の機能をまとめたモジュール 4. 手順的強度：順番に実行される複数の機能をまとめたモジュール 5. 時間的強度：ある特定の時点（プログラムの開始時など）で実行する機能をまとめたモジュール 6. 論理的強度：関連する複数の機能をまとめたモジュール 7. 暗号強度：関係のない機能をまとめたモジュール
信頼度成長曲線	-	累積バグ件数が緩やかに増えていってS字のような曲線になることが理想的
バグ管理図	-	バグ検出数：日々検出されるため、緩やかなグラフになるのが理想的 未消化テスト項目数：日々消化していくため、右肩下がりのグラフになるのが理想的 未解決バグ数：テストの進捗、バグの修正状況により増減するが、減少するのが理想的
単体テスト	-	定義した機能要件を満たすかどうか、モジュール単位でテストする

用語	フルネーム	意味
ブラックボックステスト	-	プログラムの中身を見ない（入力と出力に着目） 限界値分析：有効値と無効値の境界の値をテストする 同値分割：有効値と無効値のグループの中で、それぞれの代表的な値をテストする
ホワイトボックステスト	-	プログラムの中身に着目
命令網羅	-	すべての命令を、必ず1度は確認する。フローチャートの図形を全部通る
判定条件網羅（分析網羅）	-	すべての分岐を、少なくとも1度は確認する。フローチャートの線を全部通る
条件網羅	-	各条件式のYES/NOの組み合わせを、少なくとも1度は確認する(1+0/0+1もしくは1+1/0+0のように条件式でのテストだけで大丈夫)
複数条件網羅	-	各条件式のYes/Noの組み合わせを、全て確認する（1+1/1+0/0+1/0+0）
結合テスト	-	単体テストが完了したモジュール同士を結合して行う
トップダウンテスト	-	上⇒下に順番に結合 結合する際、まだ用意できていないモジュール（テスト中など）がある場合、ダミーデータドライバ（ボトムアップテスト）をモジュールとして用意する
ボトムアップテスト	-	下⇒上に順番に結合 結合する際、まだ用意できていないモジュール（テスト中など）がある場合、ダミーデータスタブ（トップダウンテスト）をモジュールとして用意する
総合テスト	-	テスト本番とほぼ同じ環境で、開発者がシステム全体の動作検証を行う システム全体をセキュリティや周辺環境まで含め機能観点でチェックする 機能テスト・性能テスト・負荷テスト・セキュリティテスト・ユーザアビリティテスト
受け入れテスト	-	本番とほぼ同じ環境で、ユーザーの要求通りに完成しているかユーザーがテストを行う（システム全体を業務運用観点でチェックして納品できるか判断）
リグレッションテスト 回帰テスト	-	テストで不具合を見つけたらプログラムを修正するが、その修正により、別の部分に不具合が生じる可能性がある 各テスト工程での回収後や、リリース直前、運用保守（システムを使い続けるための管理）のタイミングで行われる

テスト工程	テスト実施事項	説明・備考
単体テスト	ブラックボックステスト	内部構造は気にせず、入出力の結果のみを確認する。 テストケースは「 限界値分析・同値分析 」の考え方で作成する。
	ホワイトボックステスト	プログラムの内部構造に着目してテストを行う。 命令網羅・判定条件網羅・条件網羅・複数条件網羅 でパターンを確認する。
結合テスト	トップダウンテスト	単体テストが完了したモジュール同士を結合してテストを行う。
	ボトムアップテスト	※下位モジュールの開発が間に合わない場合は スタブ を使用する。 ※上位モジュールの開発が間に合わない場合は ドライバ を使用する。
総合テスト	機能テスト	開発者がシステム全体を機能観点でチェックする。
	性能テスト	
	負荷	
	セキュリティテスト	
	ユーザビリティテスト	
暗記不要		
受入テスト		利用者がシステム全体を業務運用観点でチェックして納品可否を判断する。

📁 システム開発の流れ

- 企業がシステムを開発する際、通常はSier等のシステム開発会社に依頼する
 - ユーザー企業との間はベンダ企業、共通フレームを使ってシステムを開発する
- 共通フレーム
 1. 【ユーザーのみで実施】企画プロセス：ニーズや課題を確認し、（システム化構想）具体的な計画を立てる（システム化計画）、RFIとRFPを実施して、ベンダ企業を選定する
 2. 要件定義プロセス：詳細機能、仕様を明確する
 3. 開発プロセス：要件をもとに設計、開発する
 4. 運用プロセス：システムを運用する
 5. 保守プロセス：問題点の修正などメンテナンスを行う
- 企画プロセス
 - RFI（Request For Information）：情報提供依頼書。一般的な情報の提供を求める（＝一次審査）
 - ベンダ企業：サービスや製品の情報提供（開発するのにふさわしいのかの判断材料）
 - RFP（Request For Proposal）：提案依頼書。個別の具体的な提案を求める（＝二次審査）
 - ベンダ企業：システム開発のスケジュール、導入計画、見積書
 - 審査通るとベンダ選定契約締結

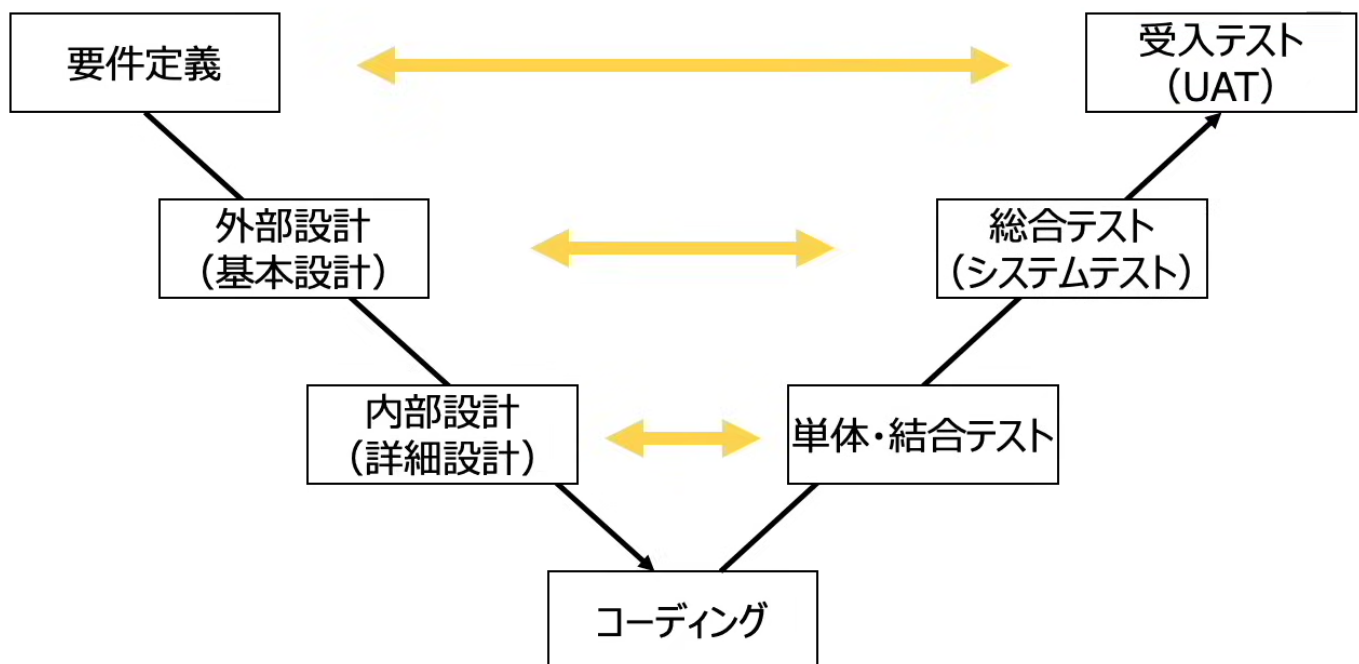


- 要件定義プロセス
 - V字モデル、設計工程とテスト工程が紐づいている
 - おさらい：
 1. 最初に要件を定義する
 2. システムを作る設計をする
 3. コーディングを行う
 4. 細かい単位でテスト

5. システム全体のテスト

6. 納品のチェック

- 下記工程の終わりにレビューを実施して、設計不備や誤りを早期に発見する
- 要件定義：必要とする機能や要求を定義
 - 機能要件：ユーザーが求めている機能
 - 非機能要件：機能以外の要件（性能、セキュリティ、保守サービスなど）
- 外部設計（基本設計）：定義された要件をシステムで実現する方法を整理してプログラマーがシステムを作るための具体的な計画書を作成する
 - システム利用者に影響する部分を設計（画面レイアウト等）
- 内部設計（詳細設計）：システム開発に影響する部分を設計（システム内のデータベース構造など）
- コーディング：プログラムを書く
- 単体・結合テスト：システムが正常に稼働するのか機能ごとにチェックしていく
 - 単体：一つの機能だけをテストする
 - 結合：複数の機能を組み合わせてテストする
- 総合テスト（システムテスト）：本番運用を想定して、システム全体の動作検証を行う（ネットワークを使用する、他システムとの連携などをチェックする）
- 受け入れテスト（UAT：User Acceptance Test）：ユーザーの要求通りに完成しているか検証する

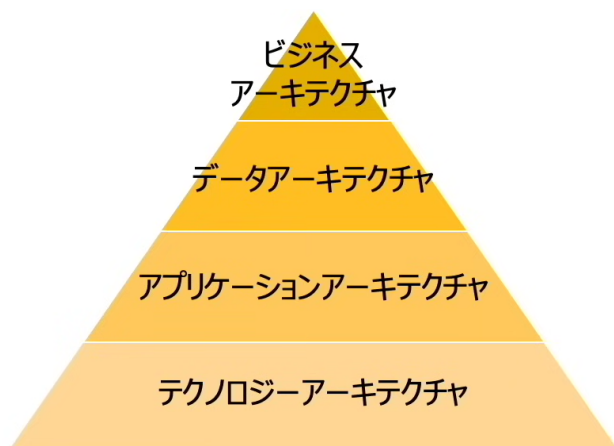


- レビュー：工程ごとに成果物の検証を行う
 - ウォークスルー（Walk Through）：担当者が関係者に対して成果物の内容を説明する
 - インスペクション（Inspection）：モデレーターが司会進行役となり、参加者の役割を明確にして進める

- ラウンドロビン（Round Robin）：参加者全員がテーマごとに持ち回りで責任者となり進める



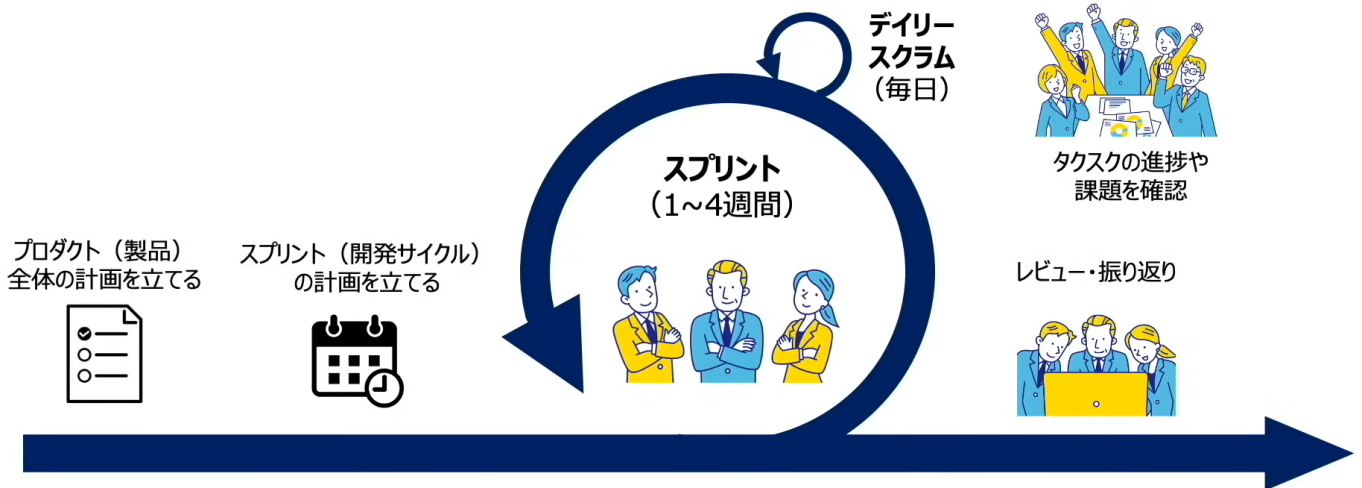
- EA(エンタープライズアーキテクチャ)：一つ一つのシステムを最適化するのではなく、**企業全体で見た時の最適化**を目指す
 - **ビジネス**アーキテクチャ
 - **データ**アーキテクチャ
 - **アプリケーション**アーキテクチャ
 - **テクノロジー**アーキテクチャ



- 企業の戦略実現に必要な業務内容、業務プロセス等
- 業務を行う上で必要なデータ、データの関連性
- データを処理する情報システムの機能
- システムの構築に必要な技術的構成要素

📖 システム開発手法

- **ウォーターフォール**：上流から下流へ順番に工程を進める
 - 要件定義・設計・開発・テストなどの工程を上流工程から下流工程へ順番に進めていく手法
 - 全体スケジュールが立てやすく、計画通りに開発を進めやすい
 - 途中の仕様変更を取り入れにくい。不具合が発生すると、後戻りが難しい
 - 活躍：仕様変更の少ない大規模のシステム開発
- **アジャイル**（機敏な、素早い）：短い開発サイクルを回す
 - 小さな機能単位ごとに開発サイクルを回し、機能ごとにリリースし段階的にシステム全体を完成させる
 - 開発期間が短い。使用変更に対応できる
 - 全体スケジュールが立てにくく、進捗が把握しづらい、開発の方法性がずれやすい
 - 活躍：要件の全体像が固まっていない小規模のシステム開発
 - XP、エクストリームプログラミング（eXtreme Programing）
 - アジャイル開発手法の一つで、顧客の要望を取り入れながら柔軟に開発を進める
 - 設計・開発・テスト・改善のサイクル（=イテレーション）を繰り返して開発する
 - 開発時の習慣的な手法5つの価値と19のプラクティスが定められている
 - 以下にて代表的のプラクティスを紹介する
- **ペアプログラミング**
 - コードを書く人（ドライバー）とコードをチェックする人（ナビゲーター）の役割に分担して、2人1組でプログラミングを行う
- **リファクタリング**
 - 外部からの挙動は変えずに、プログラムの内部構造を変更する
 - 理解しやすくにする（単純化）、バグ特定しやすくにするなど
- **テスト駆動開発**
 - プログラム実装前に先にテストコードを書いて、そのテストを通るようにコードを書く
- **スクラム**
 - アジャイル開発手法の一つで、開発チームが一体となり短期間で開発を進める
 - スプリントと呼ばれる任意から4週間程度の短い開発サイクルを回す



- **プロトタイピングモデル**：試作品を作成し、利用者がチェックを行いながら進める
 - 開発の早い段階で試作品（プロトタイプ）を作り、利用者の確認を取りながら開発を進めていく手法
 - **要件定義**で利用者の要望を聞き、システムの仕様を決めた後、見かけ上は完成品と同じである試作品を作成し、完成イメージを確認し、すりあうまで試作品を作り直す
 - 利用者との認識のズレが発生する可能性が低く、手戻りになりにくい
 - プロトタイプを作るのにコストがかかる
- **リバースエンジニアリング**：開発工程を遡り、設計書を取り出す
 - 既存の製品・プログラムを解析して、その仕組みや仕様、設計を明らかにする開発手法
 - 開発にかかるコストと時間を削減できる
 - 特許権や著作権の侵害に当たる可能性がある

📖 業務モデリングとユーザーインターフェース

- 業務モデリング

- 要件定義を行う前にはユーザー業務のヒアリングを行う
- ヒアリングした業務プロセス等業務モデルとして図式化（クラス図、シーケンス図、アクティビティ図）する
- 代表的な業務モデリング
 - E-R図：データベース設計で使用され、実体（対象業務を構成するモノ）と実体の関連をモデル化したもの
 - DFD：システムにおけるデータの流れをモデル化したもの
 - UML：オブジェクト指向のシステム開発で使用され、分析から設計、実装まで一貫した表記方法でモデル化したもの

- E-R図

- Entity-Relationship Diagram
- エンティティ：（Entity）実体（データテーブルのことを指す）
- リレーションシップ：（Relationship）関連
- データテーブルの関係性をモデル化したもの
- データベース設計で使用され、実体と実体（対象業務を構成するモノ）の関連をモデル化したもの
- リレーションシップは4種類の関連があり、矢印の有無や矢印の向きで表現される

(例) 買い上げ・入金システムのE-R図

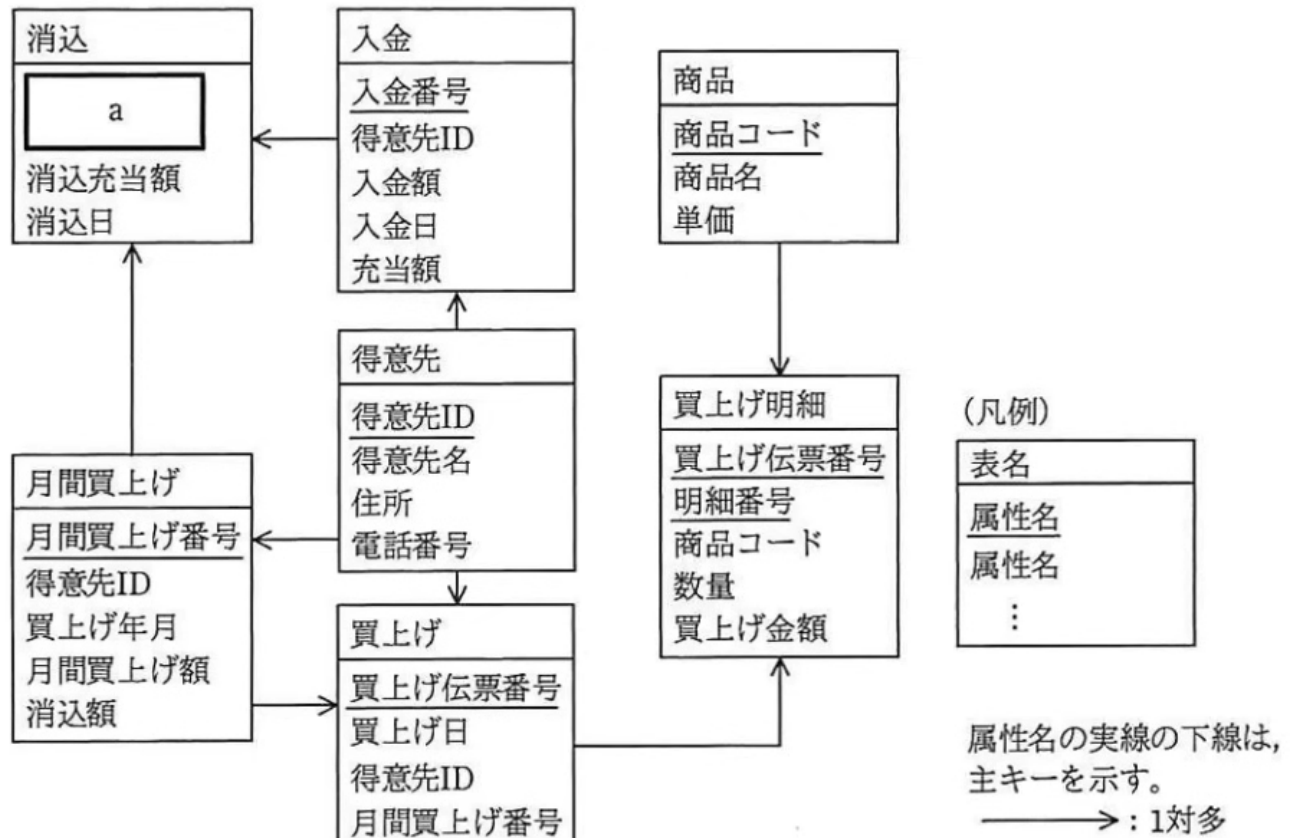
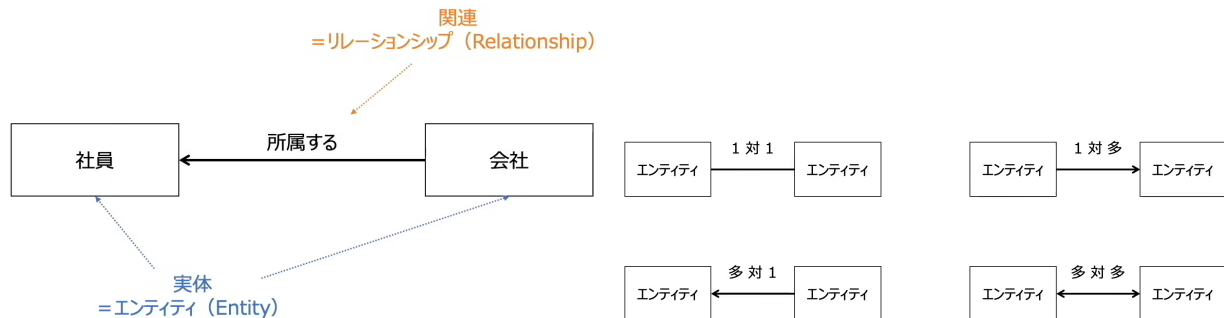
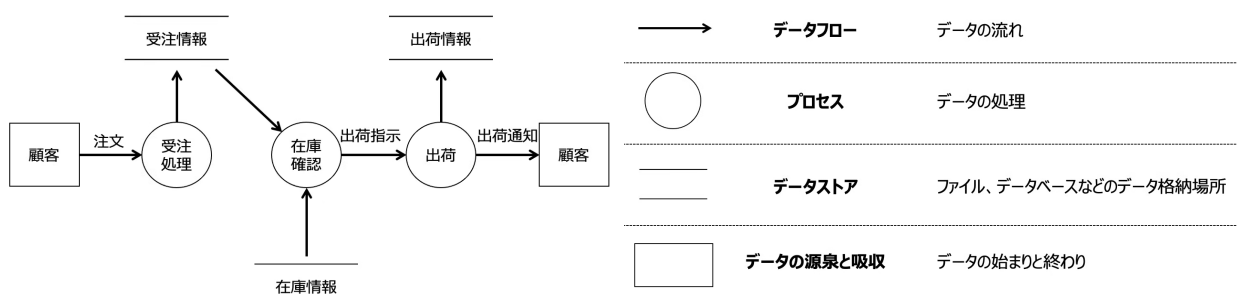


図1 関係データベースのE-R図



• DFD

- Data Flow Diagram
- システムにおけるデータの流れをモデル化したもの
- 4つの記号で記述する
 - データストア：データテーブル



• ユーザーインターフェース (UI)

- ユーザーとコンピューターの接点となるもの。ユーザーとコンピューター間で情報をやり取りする機械や入力装置
 - 外部設計の1つでありインターフェース設計と呼ばれる
 - インターフェース設計
 - 画面設計と帳票設計がある
 - 画面設計：画面の項目やレイアウトなど、データを入力する際の操作方法を設計
 - 入力順序が画面の左上から右下へ流れていくように配置
 - 操作ボタンの形や表示位置を同じにする
 - 色に同一性を保つなど
 - 帳票設計：帳簿や伝票などのレイアウト、プリンターに出力する内容などを設計
 - 関連項目を近くに配置し、余分な情報を除いて必要
 - 最小限の情報を盛り込むなど
 - 画面を操作するもの
 - GUI
 - Graphical User Interface、グラフィカルユーザーインターフェース
 - 画面上のアイコンやボタンなどをマウス操作する視覚的インターフェース
 - ラジオボタン：排他的な項目から1つを選択
 - チェックボックス：それぞれの項目で選択する/しないを指定
 - プルダウンメニュー：一覧形式のメニューから選択
 - CUI
 - Character User Interface、キャラクターユーザーインターフェース
 - キーボードによる文字入力を用いたインターフェース
 - 画像やアイコンを使わず文字だけで操作し、テキストデータのみで表現される
 - Windowsのコマンドプロンプト
 - MacOSのターミナル
-

📁 モジュール分割

- モジュール
 - システムや機器を構成する、1つの機能や要素 (=部品)
- モジュール分割
 - 内部設計の段階で行う
 - システム全体をモジュール単位で分割を行い、モジュール単位で開発・テストを行っていく
 - メリット
 - 複数人で並行して作業を進めることができる
 - 共通する機能を他のところでも再利用できる
 - プログラムに問題が起こった場合、一部の修正で解決
 - モジュールの独立性
 - 独立性とは他モジュールとの関係度合のことであり、独立性が高いとシステムとメンテナンスが行いやすい
 - モジュールの独立性が高いと、あるモジュールを変更しても、他のモジュールへの影響が少ない
 - 設計をする際、モジュール分割をする際は独立性が高くなるように設計することが大切
 - モジュールを表す指標：モジュール結合度とモジュール強度
- モジュール結合度
 - モジュール同士の関連性の強さを表す
 - モジュール結合度が弱い方が、モジュールの独立性が高くなり、システムメンテナンスしやすい
 1. データ結合：処理に必要なデータだけを引数として渡す
 2. スタンプ結合：データ構造（データのまとまり）を引数として渡す
 3. 制御結合：モジュールの実行順序を制御するデータ（制御パラメータ）を引数として渡す
 4. 外部結合：外部宣言されたデータを参照する
 5. 共通結合：外部宣言されたデータ構造（データのかたまり）を参照する
 6. 内容結合：他のモジュール内のデータを参照する

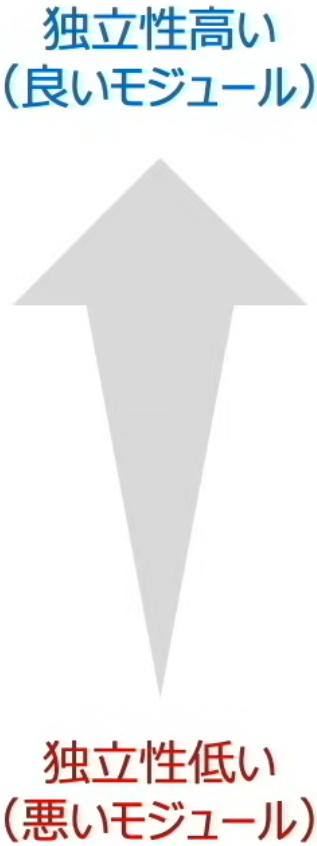
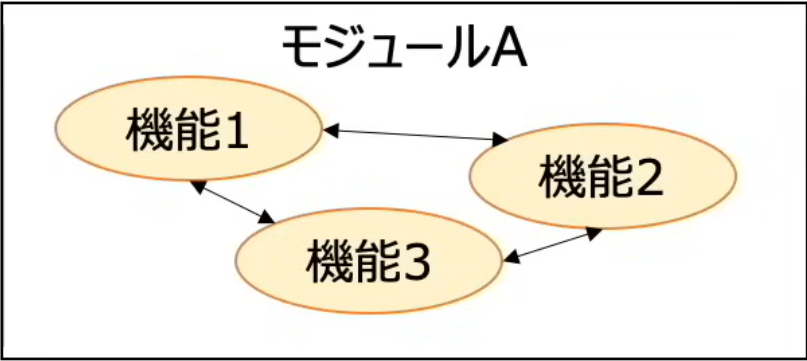
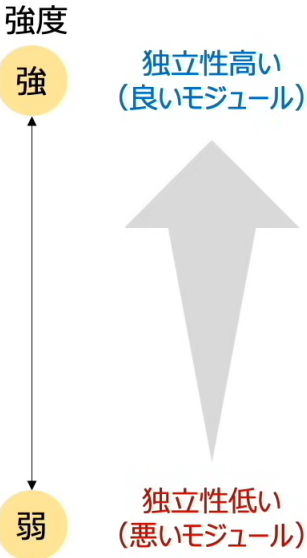
結合度	
弱	独立性高い (良いモジュール)
↑	
↓	
強	独立性低い (悪いモジュール)

データ結合	処理に必要なデータだけを引数として渡す
スタンプ結合	データ構造（データのまとまり）を引数として渡す
制御結合	モジュールの実行順序を制御するデータ（制御パラメータ）を引数として渡す
外部結合	外部宣言されたデータを参照する
共通結合	外部宣言されたデータ構造（データのまとまり）を参照する
内容結合	他のモジュール内のデータを参照する

- モジュール強度

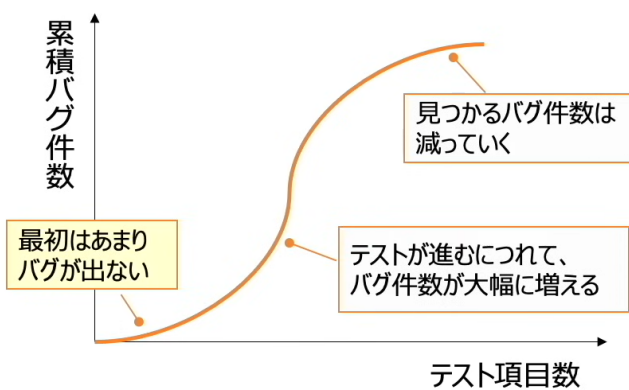
- モジュール内部の関連性の強さを表す
- モジュール強度が強い方が、モジュールの独立性が高くなり、システムメンテナンスがしやすい
- 1つのモジュールが1つの機能に洗練（強化）されているか
 - 機能的強度：一つの機能のみを持つモジュール
 - 情報強度：同じデータを使う複数の機能をまとめたモジュール
 - 連絡的強度：データの受け渡し・参照をしながら、順番に実行される複数の機能をまとめたモジュール
 - 手順的強度：順番に実行される複数の機能をまとめたモジュール
 - 時間的強度：ある特定の時点（プログラムの開始時など）で実行する機能をまとめたモジュール
 - 論理的強度：関連する複数の機能をまとめたモジュール
 - 暗号強度：関係のない機能をまとめたモジュール

機能的強度	一つの機能のみを持つモジュール
情報的強度	同じデータを使う複数の機能をまとめたモジュール
連絡的強度	データの受け渡し・参照をしながら、順番に実行される複数の機能をまとめたモジュール
手順的強度	順番に実行される複数の機能をまとめたモジュール
時間的強度	ある特定の時点（プログラムの開始時など）で実行する機能をまとめたモジュール
論理的強度	関連する複数の機能をまとめたモジュール
暗号的強度	関係のない機能をまとめたモジュール

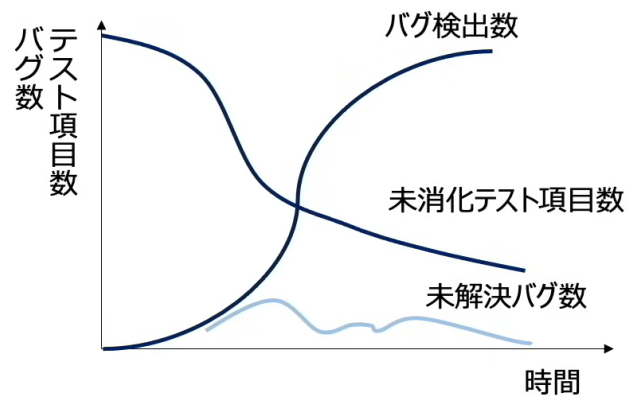


☐テスト手法

- テストとバグの関係
 - テストを行う目的は、プログラムのバグを発見し修正すること
 - 信頼度成長曲線やバグ管理図を見ることで、テストの進捗や品質を確認する
 - 信頼度成長曲線：累積バグ件数が緩やかに増えていってS字のような曲線になることが理想的
 - バグ管理図：
 - バグ検出数：日々検出されるため、緩やかなグラフになるのが理想的
 - 未消化テスト項目数：日々消化していくため、右肩下がりのグラフになるのが理想的
 - 未解決バグ数：テストの進捗、バグの修正状況により増減するが、減少するのが理想的



信頼度成長曲線（バグ曲線）

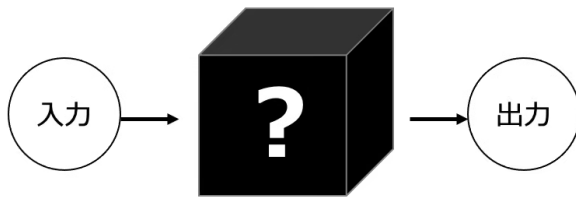


バグ管理図

- テスト工程
 - 単体・結合テスト：プログラムを構成する機能単位でテストする
- 単体テスト：定義した機能要件を満たすかどうか、モジュール単位でテストする
 - ブラックボックステスト：プログラムの中身を見ない（入力と出力に着目）
 - テストケースを作成する考え方として限界値分析と同値分割がある
 - 限界値分析：有効値と無効値の境界の値をテストする
 - 同値分割：有効値と無効値のグループの中で、それぞれの代表的な値をテストする
 - ホワイトボックステスト：プログラムの中身に着目
 - 網羅するために4パターンのテストケースがある
 - 命令網羅：すべての命令を、必ず1度は確認する。フローチャートの図形を全部通る
 - 判定条件網羅（分析網羅）：すべての分岐を、少なくとも1度は確認する。フローチャートの線を全部通る
 - 条件網羅：各条件式のYES/NOの組み合わせを、少なくとも1度は確認する(1+0/0+1もしくは1+1/0+0のように条件式でのテストだけで大丈夫)
 - 複数条件網羅：各条件式のYes/Noの組み合わせを、全て確認する（1+1/1+0/0+1/0+0）

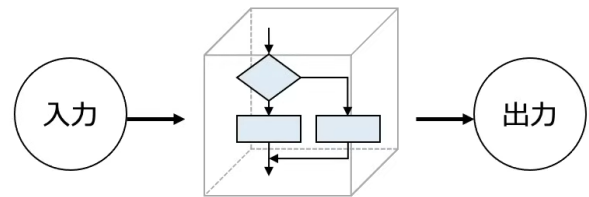
ブラックボックステスト

プログラムの中身を見ない
(=入力と出力に着目)



ホワイトボックステスト

プログラムの中身に着目

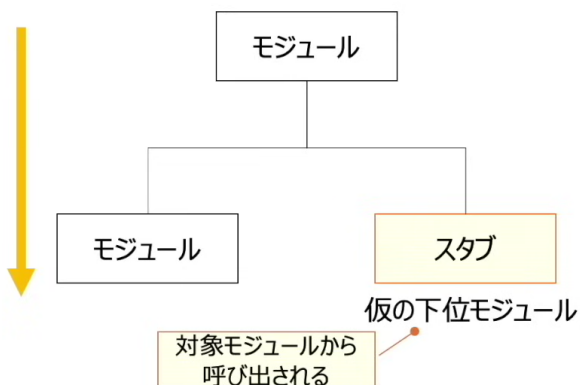


結合テスト

- 単体テストが完了したモジュール同士を結合して行う
 - **トップダウンテスト**：上⇒下に順番に結合
 - **ボトムアップテスト**：下⇒上に順番に結合
 - 結合する際、まだ用意できていないモジュール（テスト中など）がある場合、**ダミーデータスタブ**（トップダウンテスト）もしくは**ダミーデータドライバ**（ボトムアップテスト）をモジュールとして用意する

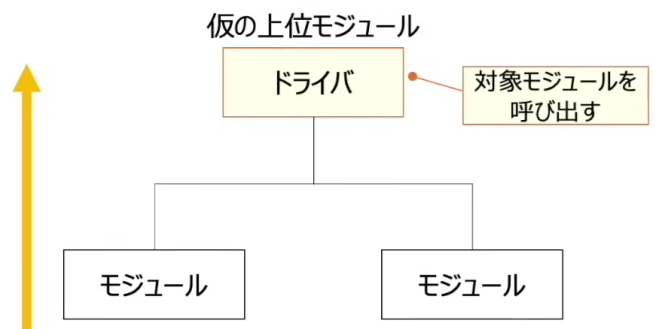
トップダウンテスト

上→下に順番に結合



ボトムアップテスト

下→上に順番に結合



総合テスト

- 本番とほぼ同じ環境で、開発者がシステム全体の動作検証を行う（システム全体をセキュリティや周辺環境まで含め機能観点でチェックする）
- 代表的なテストの種類：
 - 機能テスト：必要な機能が全て含まれており、要件通りに動作するか確認する
 - 性能テスト：システムの**処理速度**など**パフォーマンス**を確認する
 - 負荷テスト：システムに大きな負荷をかけ、**負荷がかかった状態でも**正常に動作するか確認する
 - セキュリティテスト：システムが**攻撃されても**、要件通りセキュリティを保持できるかを確認する
 - ユーザビリティテスト：操作がしやすいか、画面が見やすいかなどユーザーにとって**使いやすいかどうか**を確認する

- **受け入れテスト**：本番とほぼ同じ環境で、ユーザーの要求通りに完成しているか**ユーザー**がテストを行う（システム全体を業務運用観点でチェックして納品できるか判断）

テスト工程	テスト実施事項	説明・備考
単体テスト	ブラックボックステスト	内部構造は気にせず、入出力の結果のみを確認する。 テストケースは「 限界値分析・同値分析 」の考え方で作成する。
	ホワイトボックステスト	プログラムの内部構造に着目してテストを行う。 命令網羅・判定条件網羅・条件網羅・複数条件網羅 でパターンを確認する。
結合テスト	トップダウンテスト	単体テストが完了したモジュール同士を結合してテストを行う。
	ボトムアップテスト	※下位モジュールの開発が間に合わない場合は スタブ を使用する。 ※上位モジュールの開発が間に合わない場合は ドライバ を使用する。
総合テスト	機能テスト	開発者がシステム全体を機能観点でチェックする。
	性能テスト	
	負荷	
	セキュリティテスト	
	ユーザビリティテスト	
暗記不要		
受入テスト		利用者がシステム全体を業務運用観点でチェックして納品可否を判断する。

- リグレッションテスト（回帰テスト）
 - テストで不具合を見つけたらプログラムを修正するが、その**修正により、別の部分に不具合が生じる可能性**がある
 - 各テスト工程での回収後や、リリース直前、**運用保守**（システムを使い続けるための管理）のタイミングで行われる